

redis

Now let's provision a minimalistic Docker container that has support for Python and is based on Alpine (a security-oriented, lightweight GNU/Linux distribution). The Dockerfile for the application is provided below for reference:

```
FROM python:3.4-alpine
ADD . /code
WORKDIR /code
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

The *Docker-compose.yml* file is used to create the images and will also be used by Ansible. It defines the services that will be deployed in the containers:

```
version: '2'
services:
  web:
    build: .
    ports:
      - "5000:5000"
  redis:
    image: "redis:alpine"
    ports:
      - "6379:6379"
```

Provisioning

The Python Web application will be running on port 5000, whereas the Redis database server will be listening on port 6379. We will first build the application using the following code:

```
$ docker-compose up
```

```
Creating composetest_web_1
Creating composetest_redis_1
Attaching to composetest_web_1, composetest_redis_1
redis_1 | 1:C 05 Oct 11:40:49.067 # 0000000000000000 Redis is
starting 0000000000000000

redis_1 | 1:C 05 Oct 11:40:49.067 # Redis version=4.0.2,
bits=64, commit=00000000, modified=0, pid=1, just started

redis_1 | 1:C 05 Oct 11:40:49.067 # Warning: no config file
specified, using the default config. In order to specify a
config file use redis-server /path/to/redis.conf

redis_1 | 1:M 05 Oct 11:40:49.070 * Running mode=standalone,
port=6379.

redis_1 | 1:M 05 Oct 11:40:49.070 # WARNING: The TCP backlog
setting of 511 cannot be enforced because /proc/sys/net/core/
```

somaxconn is set to the lower value of 128.

```
redis_1 | 1:M 05 Oct 11:40:49.070 # Server initialized

redis_1 | 1:M 05 Oct 11:40:49.070 # WARNING overcommit_
memory is set to 0! Background save may fail under low memory
condition. To fix this issue add 'vm.overcommit_memory = 1' to
/etc/sysctl.conf and then reboot or run the command 'sysctl
vm.overcommit_memory=1' for this to take effect.

redis_1 | 1:M 05 Oct 11:40:49.070 # WARNING you have
Transparent Huge Pages (THP) support enabled in your kernel.
This will create latency and memory usage issues with Redis.
To fix this issue run the command 'echo never > /sys/kernel/
mm/transparent_hugepage/enabled' as root, and add it to your
/etc/rc.local in order to retain the setting after a reboot.
Redis must be restarted after THP is disabled.

redis_1 | 1:M 05 Oct 11:40:49.070 * Ready to accept
connections

web_1 | * Running on http://0.0.0.0:5000/ (Press CTRL+C
to quit)

web_1 | * Restarting with stat
web_1 | * Debugger is active!
web_1 | * Debugger PIN: 100-456-831
```

If you start a browser on the host system and open the URL *http://0.0.0.0:5000*, you will see the text from the Flask application. You can continue to refresh the page making requests to the application, and you will see the count increasing in the text: 'Hello World! I have been seen N times.' Pressing *Ctrl+c* in the above terminal will stop the application. Let's now create an Ansible playbook to launch these containers:

```
- name: Provision Flask application
  hosts: localhost
  connection: local
  become: true
  gather_facts: true
  tags: [setup]

tasks:
  - docker_service:
      project_name: composetest
      definition:
        version: '2'
        services:
          web:
            build: "{{ playbook_dir }}/."
            ports:
              - "5000:5000"
          redis:
            image: "redis:alpine"
```